

A Comparative Study on Cat-Dog Image Classification

Internship Report

Deep Learning Model Performance Analysis

Introduction

This report presents a systematic study of three critical hyperparameters in Convolutional Neural Network (CNN) training: **epochs**, **learning rate**, and **batch size**. Using a cat-dog image classification task with 8,005 images, we trained multiple models with different hyperparameter configurations to understand how each parameter affects model performance and training behavior.

1. Understanding Epochs

What is an Epoch?

An **epoch** is one complete pass through the entire training dataset. Think of it like reading a textbook: one epoch means you've read through the entire book once. In machine learning, the model needs to see the data multiple times to learn patterns effectively, just like you might need to review study material several times before an exam.

For example, if your dataset has 1,000 images and you train for 10 epochs, the model will see all 1,000 images exactly 10 times during training.

Why Multiple Epochs?

Models rarely learn everything in a single pass through the data. Each epoch allows the model to refine its internal parameters (weights) based on what it learned from the previous pass. The model improves iteratively—like practicing a skill repeatedly until you master it.

Our Experiment: 5 vs 10 vs 20 Epochs

We trained three identical CNN models with different epoch counts to see how training duration affects performance :

Results Analysis:

Epochs	Validation Accuracy	Validation Loss	Status
5	80.64%	0.4306	Underfitting - still learning
10	86.88%	0.3544	Good balance
20	89.69%	0.2525	Best performance

5 Epochs (Underfitting):

The model achieved 80.64% accuracy but was still improving rapidly when training stopped . This is like stopping your exam preparation after reading the book once—you've learned some things but haven't mastered the material. The model needed more time to find better patterns in the data.

10 Epochs (Balanced):

Accuracy jumped to 86.88%, a gain of 6.24 percentage points . This represents solid performance suitable for quick prototyping or resource-limited scenarios . The model had enough time to learn the major patterns but could still benefit from more training.

20 Epochs (Optimal):

The model reached 89.69% accuracy with the lowest validation loss (0.2525) . However, the improvement from 10 to 20 epochs was smaller (+2.81%) than from 5 to 10 epochs (+6.24%), showing **diminishing returns**. This pattern indicates the model is approaching convergence—meaning it's learned most of what it can from the data.

Key Takeaway:

More epochs generally improve accuracy, but there's a point where additional training provides minimal benefit. For our cat-dog classifier, 20 epochs appears optimal, balancing performance gains with training time .

2. Understanding Learning Rate

What is Learning Rate?

The **learning rate** controls how much the model adjusts its internal parameters (weights) after seeing each batch of data. Imagine learning to ride a bicycle: a high learning rate is like making big, aggressive corrections to your balance (fast but

unstable), while a low learning rate is like making tiny, cautious adjustments (slow but potentially stuck in bad positions).

Mathematically, learning rate determines the step size when updating weights: $\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$.

Why Learning Rate Matters

Learning rate is often called the **most important hyperparameter** in deep learning. Too high, and the model "overshoots" good solutions, bouncing around wildly. Too low, and the model learns so slowly it gets stuck in poor solutions or takes forever to train.

Our Experiment: High vs Medium vs Low Learning Rates

We tested three learning rates with the Adam optimizer over 10 epochs :

Learning Rate	Value	Validation Accuracy	Validation Loss	Training Behavior
HIGH	0.001	82.45%	0.4219	Unstable, oscillating
MEDIUM 	0.0002	83.70%	0.3657	Smooth, stable
LOW 	0.00001	64.90%	0.7031	Too slow, stuck

Detailed Analysis:

HIGH Learning Rate (0.001) - Unstable Training:

The model's validation accuracy oscillated wildly throughout training:

- Epoch 4: Dropped to 52.40% with loss spiking to 2.0039
- Epoch 5: Fell further to 48.97% (worse than random guessing!)
- Epoch 9: Suddenly jumped to 82.45% (best epoch)
- Epoch 10: Dropped back to 74.33%

What happened? The large learning rate caused the model to make huge weight updates, "jumping around" the solution space instead of smoothly descending toward the optimal solution. It's like trying to land a ball in a cup by throwing it hard—sometimes you get lucky and it goes in, but most of the time it bounces out.

While the model eventually reached 82.45%, the unpredictable behavior makes it unreliable for production use . You can't trust that training will succeed every time.

MEDIUM Learning Rate (0.0002) - Optimal Performance: ★

This learning rate produced the **best and most stable** results :

- Smooth, consistent improvement from epoch 4 onward
- No catastrophic loss spikes or accuracy drops
- Best final accuracy (83.70%) and lowest loss (0.3657)
- Predictable training behavior

The validation accuracy progression shows healthy learning:

- Epochs 1-3: Slow start while exploring the initial weights (49-50%)
- Epoch 4: Breakthrough moment (71.96% - found good descent direction)
- Epochs 5-9: Steady improvement (57% → 83.70%)
- Epoch 10: Minor overfitting detected (dropped to 80.39%)

This is the "Goldilocks" learning rate—**not too fast, not too slow, just right**. The medium learning rate (0.0002) is the industry standard for Adam optimizer with CNNs of this size.

LOW Learning Rate (0.00001) - Failed to Learn: ✗

This learning rate was catastrophically slow :

- After 10 full epochs, reached only 64.90% accuracy (18.8% worse than medium LR)
- Barely improved from random-guess baseline (~50%)
- Got stuck at 64.90% on epoch 6 and never recovered
- Would need 50-100+ epochs to reach comparable performance

Why did it fail? The weight updates were too small to escape poor initial conditions. With LR = 0.00001, each gradient update changes weights by only 0.000001 to 0.00001—tiny movements that don't significantly change predictions . The model made microscopic steps when it needed to take meaningful strides.

Even when the ReduceLROnPlateau callback detected stagnation and reduced the learning rate further (epoch 9), it couldn't help because the learning rate was already too small.

Key Insight:

A 20× difference in learning rate (0.001 vs 0.00001) caused a **28.7% difference in**

accuracy (82.45% vs 64.90%) . Learning rate is the single most impactful hyperparameter—getting it wrong can make or break your model.

3. Understanding Batch Size

What is Batch Size?

Batch size determines how many training images the model processes before updating its weights. Instead of using all 6,404 training images at once (which would be computationally expensive), we divide them into smaller "batches".

For example, with batch size 32, the model processes 32 images, calculates the average error, updates weights, then moves to the next 32 images. With 6,404 training images and batch size 32, one epoch requires 201 weight updates ($6404 \div 32 \approx 201$).

How Batch Size Affects Training

Small batches (e.g., 16):

- More frequent weight updates per epoch (401 updates)
- Noisier gradients (each batch is less representative of the full dataset)
- Better generalization but less stable training
- Lower GPU memory usage

Large batches (e.g., 128):

- Fewer weight updates per epoch (51 updates)
- Smoother, more accurate gradients
- Risk of converging to "sharp minima" that generalize poorly
- Higher GPU memory usage

Our Experiment: Batch Sizes 16 vs 32 vs 128

We trained three models with different batch sizes for 10 epochs :

Batch Size	Steps per Epoch	Validation Accuracy	Validation Loss	Result
Low (16)	401	78.45%	0.4713	Good, slightly unstable
Medium (32) ✔	201	80.57%	0.4740	Best performer
High (128) ✖	51	49.22%	0.8713	Complete failure

Detailed Analysis:

Batch Size 16 - Second Place:

Performance: 78.45% validation accuracy

With 401 weight updates per epoch, this model had the most opportunities to adjust its weights. The frequent updates acted as implicit regularization due to gradient noise, helping the model explore the solution space effectively.

However, training showed some instability—validation accuracy peaked at epoch 7 (78.45%) then dropped in subsequent epochs, triggering early stopping. This volatility is characteristic of small batch sizes.

Batch Size 32 - Optimal Choice: ★

Performance: 80.57% validation accuracy (BEST)

This batch size achieved the perfect balance :

- Smooth, steady improvement throughout all 10 epochs
- No early stopping needed—kept improving until the end
- Minimal train-validation gap (4.27%), indicating healthy generalization
- Industry-standard batch size proven effective

The model had 201 weight updates per epoch—enough to explore the loss landscape effectively while maintaining gradient stability. This is why batch size 32 is the default choice for many deep learning practitioners.

Batch Size 128 - Catastrophic Failure: ❌

Performance: 49.22% validation accuracy (random guessing!)

This model **completely failed to learn**, getting stuck at approximately 50% accuracy throughout all 10 epochs. For a binary classification task (cats vs dogs), 50% is what you'd get by randomly guessing—the model learned nothing useful .

Why did it fail so badly?

1. **Too few weight updates:** Only 51 updates per epoch wasn't enough to explore the parameter space
2. **Sharp minimizer convergence:** Large batches create very smooth gradients that confidently converge to the nearest minimum, even if it's a poor solution
3. **Learning rate mismatch:** The learning rate (0.0002) was calibrated for batch size 32; large batches typically need proportionally larger learning rates

The model essentially learned to predict one class for almost all images—a trivial solution that achieved 49% accuracy . Even when the ReduceLROnPlateau callback reduced the learning rate twice (epochs 4 and 9), it couldn't escape this poor solution.

Key Learning:

Bigger is NOT always better in deep learning. Batch size 128 performed **31% worse** than batch size 32, demonstrating that default hyperparameters must be carefully selected based on architecture and dataset size .

Comparative Summary

Performance Comparison Table:

Hyperparameter	Configuration	Validation Accuracy	Validation Loss	Training Characteristics
Epochs	5 epochs	80.64%	0.4306	Underfitting, needs more time
	10 epochs	86.88%	0.3544	Good balance
	20 epochs 	89.69%	0.2525	Optimal performance
Learning Rate	HIGH (0.001)	82.45%	0.4219	Unstable, oscillating
	MEDIUM (0.0002) 	83.70%	0.3657	Stable, smooth convergence
	LOW (0.00001)	64.90%	0.7031	Too slow, failed to learn
Batch Size	Low (16)	78.45%	0.4713	Good, slightly unstable
	Medium (32) 	80.57%	0.4740	Best balance
	High (128)	49.22%	0.8713	Complete failure

Key Findings

1. Epochs: More is Better, But Diminishing Returns

Training for more epochs consistently improved accuracy :

- 5→10 epochs: **+6.24%** improvement (strong gains)
- 10→20 epochs: **+2.81%** improvement (diminishing returns)

The model benefits from extended training but approaches convergence around 20 epochs. For production models, training for 20-30 epochs with early stopping provides optimal results .

2. Learning Rate: The Most Critical Hyperparameter

Learning rate had the **strongest impact** on model performance:

- A 10× reduction (0.0002 → 0.00001) caused a **28.7% accuracy drop**
- Medium LR (0.0002) provided both best accuracy AND most stable training
- Default Adam LR (0.001) caused training instability in our architecture

Lesson: Always spend time finding the right learning rate before investing in longer training.

3. Batch Size: Balance Between Speed and Generalization

Medium batch size (32) outperformed both extremes :

- Batch 16: Good generalization but slightly unstable (78.45%)
- Batch 32: **Optimal balance** (80.57%)
- Batch 128: **Catastrophic failure** (49.22%)

Large batch sizes require careful hyperparameter tuning (especially learning rate scaling) to avoid convergence to poor solutions.

Optimal Configuration Recommendation

Based on our systematic experiments, the **optimal hyperparameter configuration** for this cat-dog CNN classifier is :

Configuration:

- **Epochs:** 20 (with early stopping, patience=6)
- **Learning Rate:** 0.0002 (for Adam optimizer)

- **Batch Size:** 32
- **Expected Accuracy:** ~90%

Rationale:

- This combination maximizes accuracy while maintaining stable, predictable training
- All three parameters are at their optimal values based on empirical testing
- The configuration balances training speed with final performance
- Suitable for production deployment with consistent results